1. Постановка задачи.

Разработайте алгоритм для робота-уборщика, который должен очистить все комнаты в квартире с разным типом покрытия (ковер, плитка, паркет). Учтите, что:

На ковре нужно использовать только сухую уборку.

На мокрой плитке робот может поскользнуться (проверка датчика).

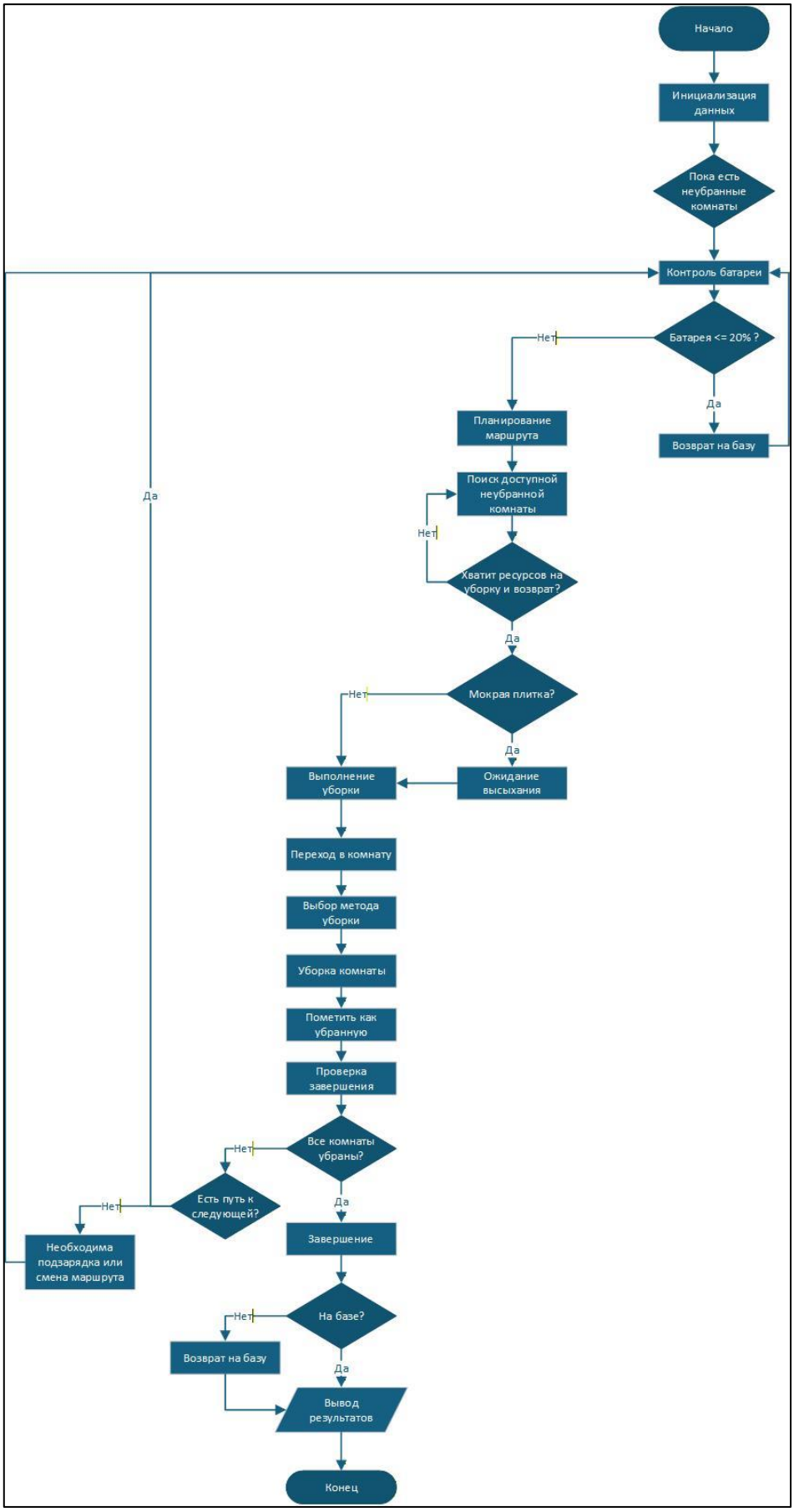
В одну комнату можно заехать только из определенных других (задана матрица смежности комнат).

У робота ограничен заряд батареи, при достижении 20% он должен вернуться на базу.

2. Контрольный пример.

Входные данные	**Значения**
rooms	"База", "Гостиная", "Кухня", "Спальня", "Ванная"
floorTypes	"-", "паркет", "плитка", "ковер", "плитка"
battery	100
Выходные данные	**Значения**
Сообщение	"Все комнаты убраны!", "Внимание: заряд ниже 20%. Возвращаемся на базу.", "Внимание: плитка мокрая! Пропускаем уборку.", "Уборка завершена."

3. Блок схема.


Начало
Инициализация данных
Пока есть неубранные комнаты
Контроль батареи
Батарея <= 20% ?
Нет
Да
Возврат на базу
Планирование маршрута
Поиск доступной неубранной комнаты
Нет
Хватит ресурсов на уборку и возврат?
Да
Мокрая плитка?
Нет
Да
Ожидание высыхания
Выполнение уборки
Переход в комнату
Выбор метода уборки
Уборка комнаты
Пометить как убранную
Проверка завершения
Все комнаты убраны?
Нет
Да
Есть путь к следующей?
Нет
Да
Необходима подзарядка или смена маршрута
Завершение
На базе?
Нет
Да
Возврат на базу
Вывод результатов
Конец

4. Текст программы.

1. Постановка задачи.

Составьте алгоритм анализа свидетельских показаний по делу. На входе: N показаний, каждое содержит время, место, описание человека. Алгоритм должен:

1) Отфильтровать противоречивые показания (разное описание для одного времени и места).

2) Выделить "ядро" непротиворечивых показаний.

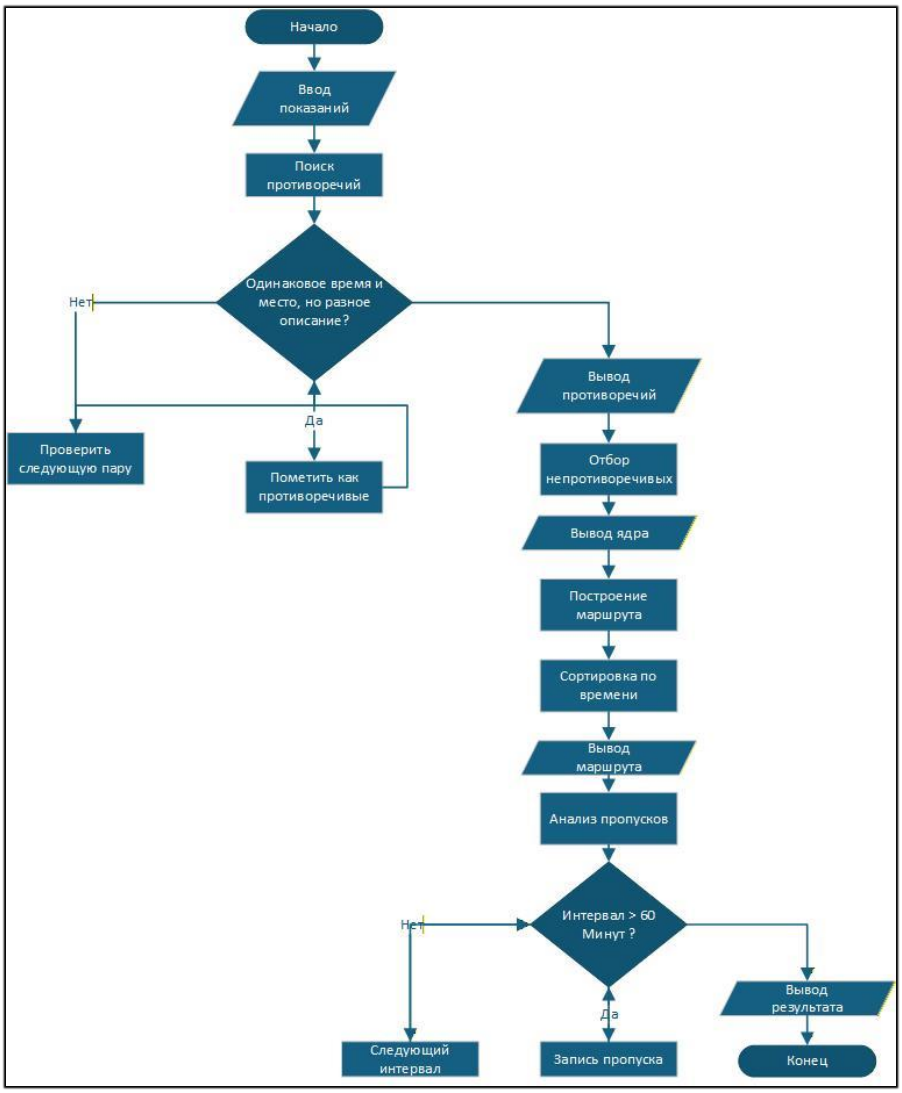
3) Построить вероятный маршрут подозреваемого.

4) Определить, есть ли в показаниях "белые пятна" (пропуски во времени), требующие дополнительного расследования.

2. Контрольный пример.

Входные данные	**Значения**
allTimes, allPlaces, allDescriptions	Массивы строк (20 элементов)
n	10
Выходные данные	**Значения**
consistent	Список непротиворечивых показаний
gapFound	True/False

3. Блок схема.



4. Текст программы.

```
Console.WriteLine($"Случайно выбранные показания ({n} из 20):");
for (int i = 0; i < n; i++)
{
    Console.WriteLine($"{i + 1}. {times[i]}, {places[i]} - {descriptions[i]}");
}
Console.WriteLine();

for (int i = 0; i < n; i++)
{
    for (int j = i + 1; j < n; j++)
    {
        if (times[i] == times[j] && places[i] == places[j] && descriptions[i] != descriptions[j])
        {
            consistent[i] = false;
            consistent[j] = false;
            Console.WriteLine($"Противоречие: показания {i + 1} и {j + 1}");
            Console.WriteLine($"  Время: {times[i]}, место: {places[i]}");
            Console.WriteLine($"  Показание {i + 1}: {descriptions[i]}");
            Console.WriteLine($"  Показание {j + 1}: {descriptions[j]}");
        }
    }
}

Console.WriteLine("\nЯдро непротиворечивых показаний:");
int count = 0;
for (int i = 0; i < n; i++)
{
    if (consistent[i])
    {
        Console.WriteLine($"{times[i]}, {places[i]} - {descriptions[i]}");
        count++;
    }
}

Console.WriteLine("\nВероятный маршрут подозреваемого:");
for (int i = 0; i < n; i++)
{
    if (consistent[i])
    {
        Console.WriteLine($"{times[i]} - {places[i]}");
    }
}

Console.WriteLine("\nПроверка временных пропусков:");
bool gapFound = false;
for (int i = 0; i < n - 1; i++)
{
    if (consistent[i] && consistent[i + 1])
    {
        int t1 = Convert.ToInt32(times[i].Replace(":", ""));
        int t2 = Convert.ToInt32(times[i + 1].Replace(":", ""));
        int diff = t2 - t1;
        if (diff > 40)
        {
            Console.WriteLine($"Пропуск {diff} минут между {times[i]} ({places[i]}) и {times[i + 1]} ({places[i + 1]})");
            gapFound = true;
        }
    }
}

if (!gapFound)
{
    Console.WriteLine("Временных пропусков не обнаружено.");
}

Console.WriteLine("\nИтог:");
Console.WriteLine($"Непротиворечивых показаний: {count} из {n}");
if (count == n)
{
    Console.WriteLine("Все показания согласованы.");
}
else if (count >= n - 2)
{
    Console.WriteLine("Обнаружены отдельные противоречия. Требуется допрос свидетелей.");
}
else
{
    Console.WriteLine("Слишком много противоречий. Показания недостоверны.");
}
```

1. Постановка задачи.

Создайте блок-схему системы, которая анализирует поток текстовых отзывов о товаре.

Шаг 1: Предобработка (удаление стоп-слов, лемматизация).

Шаг 2: Определение тональности (позитив, негатив, нейтраль) с использованием словаря эмоциональных слов и оценок.

Шаг 3: Выделение ключевых сущностей (например, "батарея", "экран", "доставка").

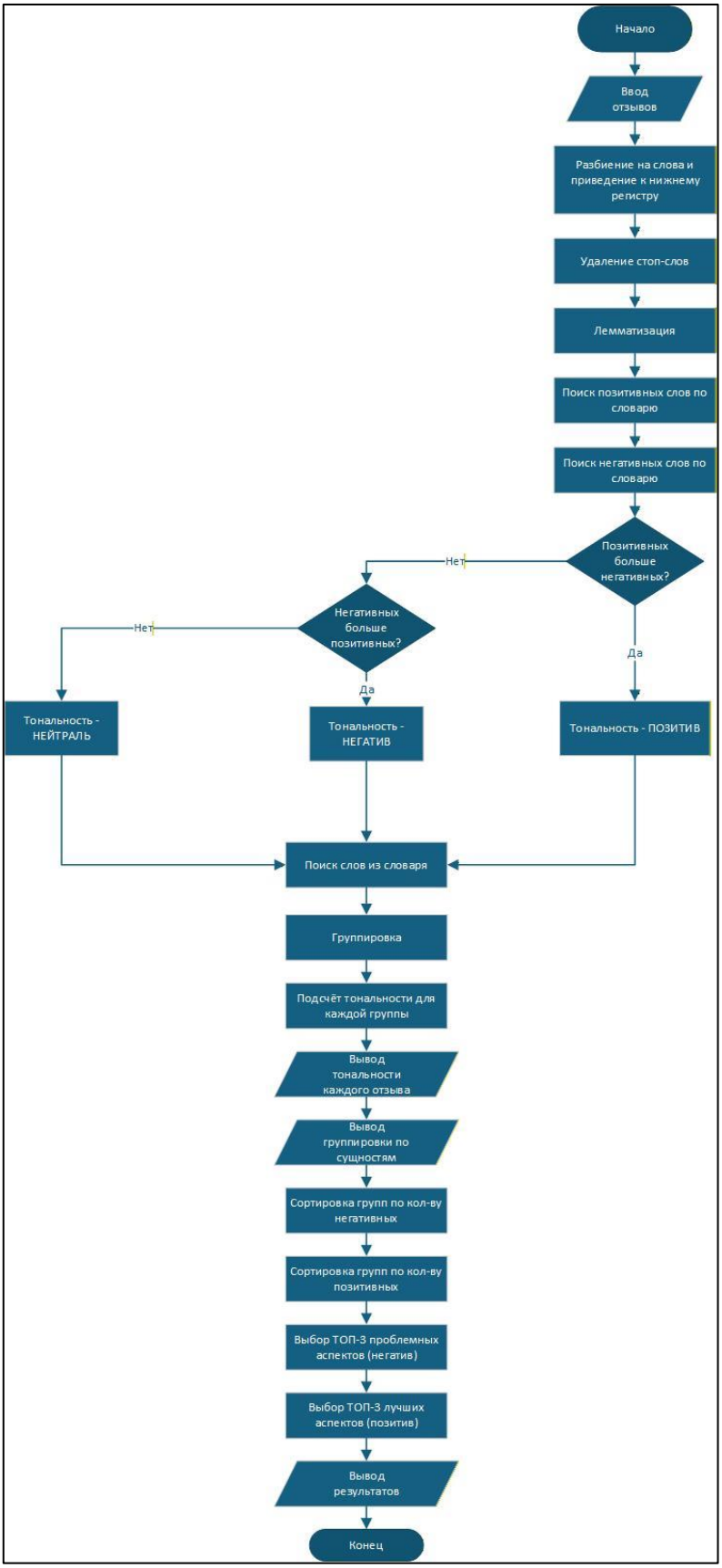
Шаг 4: Группировка отзывов по сущностям и подсчет соотношения тональности для каждой.

Шаг 5: Формирование итогового отчета: ТОП-3 проблемных и ТОП-3 лучших аспектов товара.

2. Контрольный пример.

Входные данные	**Значения**
otz	Массив из 7 строк с отзывами
ent	"батарея", "экран", "доставка", "телефон"
Выходные данные	**Значения**
pos/neg	Массивы счетчиков для каждого аспекта
ТОП проблем	Список из 3 аспектов с наибольшим числом негативных отзывов
ТОП достоинств	Список из 3 аспектов с наибольшим числом позитивных отзывов

3. Блок схема.


Начало
Ввод отзывов
Разбиение на слова и приведение к нижнему регистру
Удаление стоп-слов
Лемматизация
Поиск позитивных слов по словарю
Поиск негативных слов по словарю
Позитивных больше негативных?
Нет
Да
Негативных больше позитивных?
Нет
Да
Тональность - НЕЙТРАЛЬ
Тональность - НЕГАТИВ
Тональность - ПОЗИТИВ
Поиск слов из словаря
Группировка
Подсчёт тональности для каждой группы
Вывод тональности каждого отзыва
Вывод группировки по сущностям
Сортировка групп по кол-ву негативных
Сортировка групп по кол-ву позитивных
Выбор ТОП-3 проблемных аспектов (негатив)
Выбор ТОП-3 лучших аспектов (позитив)
Вывод результатов
Конец

4. Текст программы.

```
Console.WriteLine("Анализ отзывов:");

for (int i = 0; i < otz.Length; i++)
{
    string t = otz[i].ToLower();
    Console.WriteLine($"\nОтзыв {i + 1}: {t}");

    bool p = false;
    bool n = false;

    if (t.Contains("отличн") || t.Contains("хорош") || t.Contains("супер") || t.Contains("быстр"))
        p = true;

    if (t.Contains("ужасн") || t.Contains("плох") || t.Contains("треснул") || t.Contains("недоволен") || t.Contains("садится") || t.Contains("тухнет"))
        n = true;

    for (int j = 0; j < ent.Length; j++)
    {
        if (t.Contains(ent[j]))
        {
            if (p) pos[j]++;
            if (n) neg[j]++;
            Console.WriteLine($"  Нашли: {ent[j]} - позитив: {p}, негатив: {n}");
        }
    }
}

Console.WriteLine("\nРезультаты по аспектам:");
for (int i = 0; i < ent.Length; i++)
{
    Console.WriteLine($"{ent[i]}: позитив = {pos[i]}, негатив = {neg[i]}");
}

Console.WriteLine("\nТОП проблем:");
int max1 = -1, max2 = -1, max3 = -1;
int i1 = -1, i2 = -1, i3 = -1;
for (int i = 0; i < ent.Length; i++)
{
    if (neg[i] > max1)
    {
        max3 = max2; i3 = i2;
        max2 = max1; i2 = i1;
        max1 = neg[i]; i1 = i;
    }
    else if (neg[i] > max2)
    {
        max3 = max2; i3 = i2;
        max2 = neg[i]; i2 = i;
    }
    else if (neg[i] > max3)
    {
        max3 = neg[i]; i3 = i;
    }
}
if (i1 != -1)
    Console.WriteLine($"1. {ent[i1]} ({neg[i1]} негативных)");
if (i2 != -1)
    Console.WriteLine($"2. {ent[i2]} ({neg[i2]} негативных)");
if (i3 != -1)
    Console.WriteLine($"3. {ent[i3]} ({neg[i3]} негативных)");

Console.WriteLine("\nТОП достоинств:");

max1 = -1; max2 = -1; max3 = -1;
i1 = -1; i2 = -1; i3 = -1;
for (int i = 0; i < ent.Length; i++)
{
    if (pos[i] > max1)
    {
        max3 = max2; i3 = i2;
        max2 = max1; i2 = i1;
        max1 = pos[i]; i1 = i;
    }
    else if (pos[i] > max2)
    {
        max3 = max2; i3 = i2;
        max2 = pos[i]; i2 = i;
    }
    else if (pos[i] > max3)
    {
        max3 = pos[i]; i3 = i;
    }
}

if (i1 != -1)
    Console.WriteLine($"1. {ent[i1]} ({pos[i1]} позитивных)");
if (i2 != -1)
    Console.WriteLine($"2. {ent[i2]} ({pos[i2]} позитивных)");
if (i3 != -1)
    Console.WriteLine($"3. {ent[i3]} ({pos[i3]} позитивных)");
```

1. Постановка задачи.

Смоделируйте алгоритм распределения ограниченного бюджета между несколькими IT-проектами на основе:

1) Приоритета от заказчика (высокий/средний/низкий).

2) Предварительной оценки стоимости.

3) Рассчитанного ROI (возврат на инвестиции).

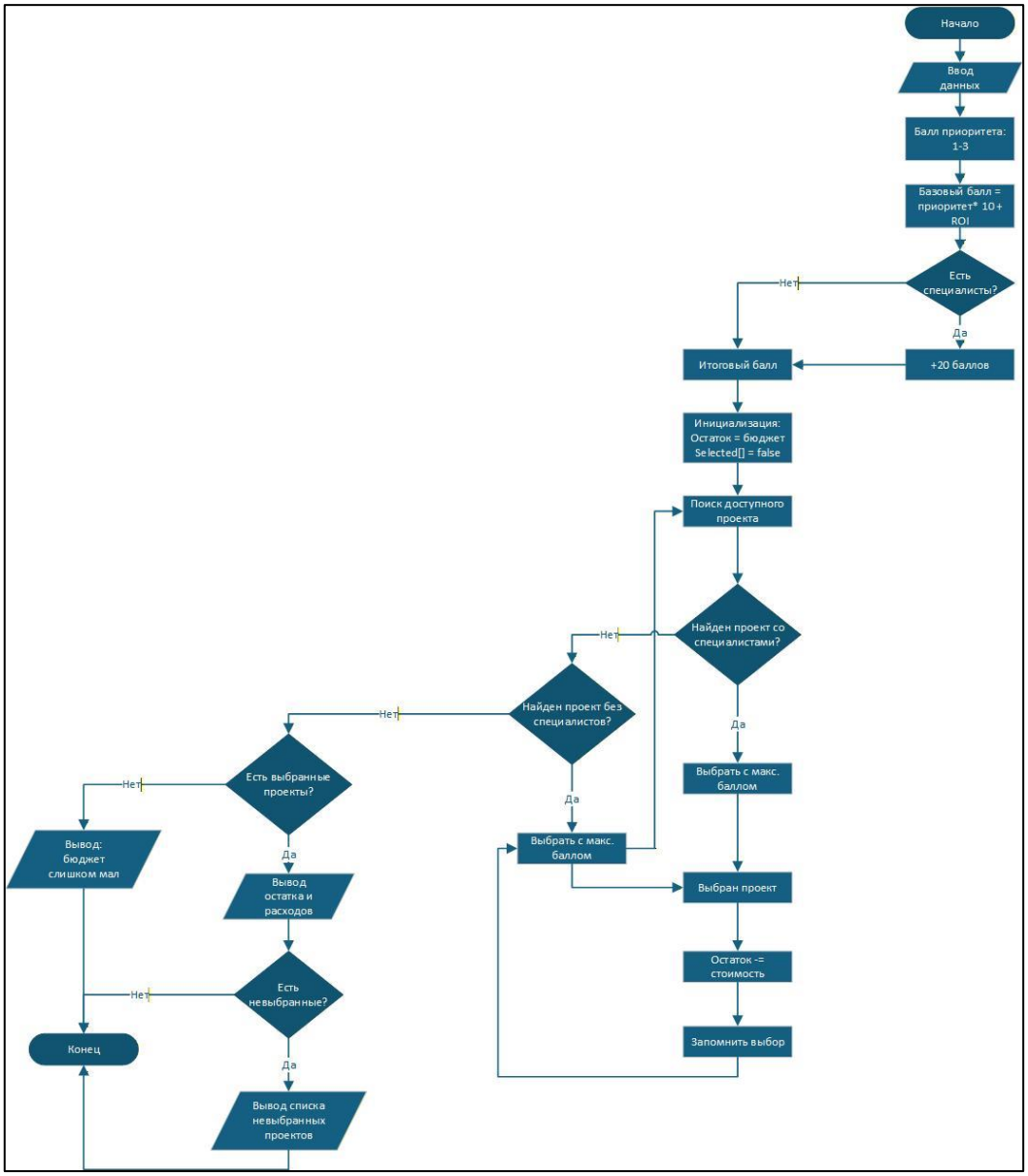
4) Наличия необходимых специалистов в команде.

Алгоритм должен итеративно подбирать оптимальный портфель проектов, пока бюджет не будет исчерпан или не кончатся подходящие проекты. Включите ветвление для случаев, когда бюджет слишком мал для любого проекта.

2. Контрольный пример.

Входные данные	**Значения**
names	"Сайт", "Мобильное приложение", "Аналитика данных", "Чат-бот", "Облачное хранилище"
priority	3, 2, 1, 2, 3
cost	50000, 80000, 30000, 40000, 100000
roi	1.5, 2.0, 1.2, 1.8, 2.5
hasSpecialists	true, true, false, true, false
budget	Вводится пользователем
Выходные данные	**Значения**
selected	Список выбранных проектов
totalSelected	Количество выбранных проектов
remaining	Остаток бюджета

3. Блок схема.


Начало
Ввод данных
Балл приоритета: 1-3
Базовый балл = приоритет* 10 + ROI
Есть специалисты?
Нет
Да
+20 баллов
Итоговый балл
Инициализация: Остаток = бюджет Selected[] = false
Поиск доступного проекта
Найден проект со специалистами?
Нет
Да
Выбрать с макс. баллом
Найден проект без специалистов?
Нет
Да
Выбрать с макс. баллом
Выбран проект
Остаток -= стоимость
Запомнить выбор
Есть выбранные проекты?
Нет
Да
Вывод: бюджет слишком мал
Вывод остатка и расходов
Есть невыбранные?
Нет
Да
Вывод списка невыбранных проектов
Конец

4. Текст программы.

```
// Проверка минимального бюджета
int minCost = 1000000;
for (int i = 0; i < 5; i++)
{
    if (hasSpecialists[i] && cost[i] < minCost)
        minCost = cost[i];
}

if (budget < minCost)
{
    Console.WriteLine("\nБюджет слишком мал для запуска любого проекта.");
    return;
}

Console.WriteLine("\nОтбор проектов по приоритету и доступности...\n");

for (int p = 3; p >= 1; p--)
{
    for (int i = 0; i < 5; i++)
    {
        if (priority[i] == p && hasSpecialists[i] && !selected[i])
        {
            if (cost[i] <= remaining)
            {
                selected[i] = true;
                remaining -= cost[i];
                totalSelected++;
                Console.WriteLine($"☑ {names[i]} | Приоритет: {p}, Стоимость: {cost[i]}, ROI: {roi[i]}x");
            }
        }
    }
}

Console.WriteLine($"\nИтог:");
Console.WriteLine($"Выбрано проектов: {totalSelected}");
Console.WriteLine($"Использовано средств: {budget - remaining} руб");
Console.WriteLine($"Остаток бюджета: {remaining} руб");

if (totalSelected == 0)
{
    Console.WriteLine("Нет проектов, которые можно запустить в рамках бюджета.");
}
else
{
    Console.WriteLine("Распределение ресурсов завершено.");
}
```

1. Постановка задачи.

Разработайте алгоритм управления сложным перекрестком (с полосой для общественного транспорта и пешеходным переходом). Условия:

Основной цикл работает по таймеру.

Датчик общественного транспорта имеет высший приоритет и запускает специальную фазу.

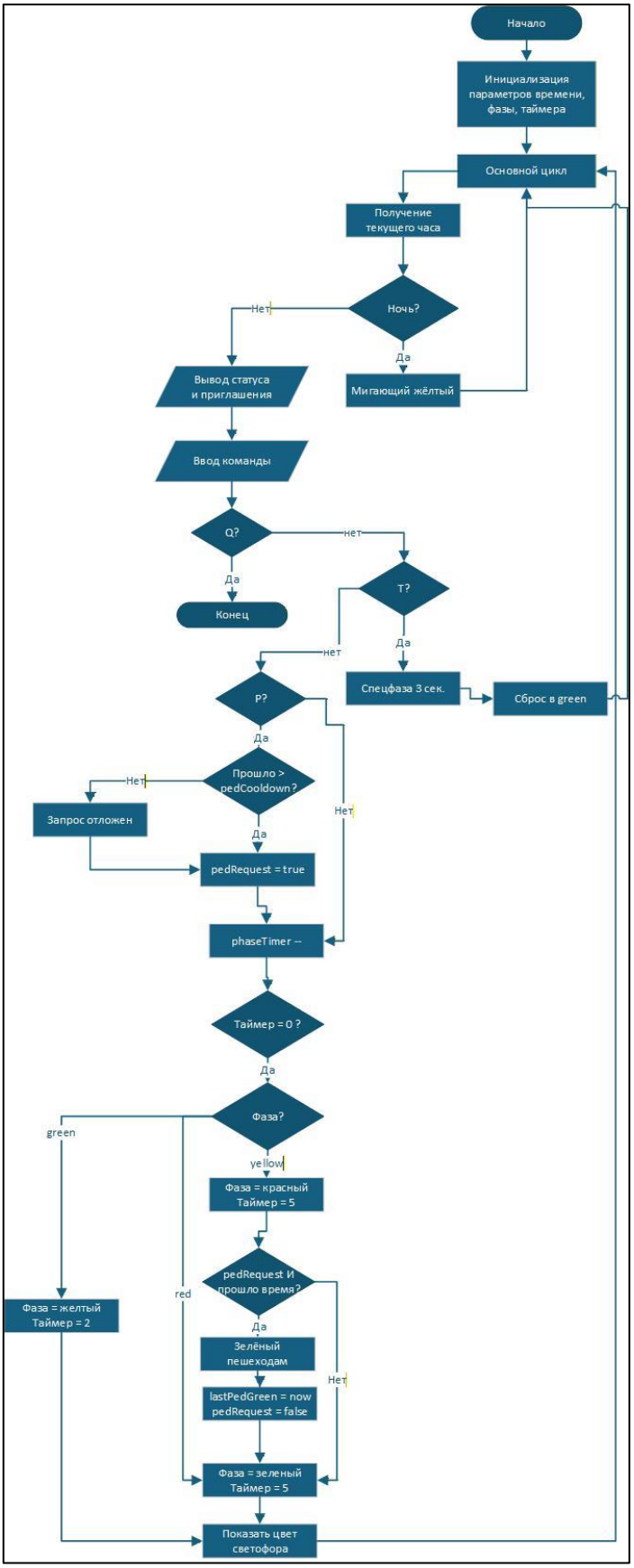
Датчик пешеходов нажимается кнопкой, но запрос обрабатывается не сразу, а в следующем допустимом цикле, если с момента последнего включения прошло больше N секунд (защита от "досрочного" нажатия).

В ночное время режим переключается на мигающий желтый.

2. Контрольный пример.

Входные данные	**Значения**
hour	Текущий час (0–23)
cycles	3
rand.Next(100) < 25	Вероятность запроса пешехода (25%)
rand.Next(100) < 15	Вероятность появления автобуса (15%)
Выходные данные	**Значения**
Режим	"Ночное время. Режим: мигающий желтый сигнал." или "Дневной режим работы."
События	"Приоритет: обнаружен общественный транспорт.", "Разрешен запрос пешехода.", "Стандартная фаза..."

3. Блок схема.


Начало
Инициализация параметров времени, фазы, таймера
Основной цикл
Получение текущего часа
Ночь?
Нет
Да
Вывод статуса и приглашения
Мигающий жёлтый
Ввод команды
Q?
нет
Да
Конец
T?
нет
Да
Спецфаза 3 сек.
Сброс в green
P?
Да
Прошло > pedCooldown?
Нет
Нет
Запрос отложен
Да
pedRequest = true
phaseTimer --
Таймер = 0 ?
Да
Фаза?
green
yellow
red
Фаза = красный Таймер = 5
pedRequest И прошло время?
Фаза = желтый Таймер = 2
Да
Нет
Зелёный пешеходам
lastPedGreen = now pedRequest = false
Фаза = зеленый Таймер = 5
Показать цвет светофора

4. Текст программы.

```
for (int cycle = 0; cycle < cycles; cycle++)
{
    Console.WriteLine($"--- Цикл {cycle + 1} ---");

    if (!pedRequest && rand.Next(100) < 25)
    {
        pedRequest = true;
        Console.WriteLine("Обнаружен запрос пешехода (нажата кнопка).");
    }

    bool bus = (rand.Next(100) < 15);
    if (bus)
    {
        Console.WriteLine(" ! Приоритет: обнаружен общественный транспорт.");
        Console.WriteLine("Включена специальная фаза для автобуса (зеленый 10 сек).");
        Console.WriteLine("Пешеходный запрос отложен.");
        continue;
    }

    if (pedRequest && (cycle - lastPedTime) >= 2)
    {
        Console.WriteLine("☑ Разрешен запрос пешехода.");
        Console.WriteLine("Зеленый для пешеходов (15 сек), красный для автомобилей.");
        pedRequest = false;
        lastPedTime = cycle;
    }
    else
    {
        Console.WriteLine("Стандартная фаза:");
        Console.WriteLine("Зеленый для автомобилей (20 сек) → Желтый (3 сек) → Красный (10 сек)");
        if (pedRequest)
            Console.WriteLine("(Запрос пешехода ожидает следующего допустимого цикла)");
    }
}

Console.WriteLine("\nРабота светофора завершена.");
```

1. Постановка задачи.

Постройте блок-схему, которая по данным клиента (возраст, доход, ежемесячные траты, сумма сбережений, цель: "подушка безопасности", "покупка жилья", "пенсия") предлагает финансовый план.

Алгоритм должен:

1) Рассчитать "свободные" средства.

2) Оценить риск-профиль на основе возраста и опыта.

3) Предложить распределение активов (вклад, фонды, акции, облигации) в процентах в зависимости от цели и риска.

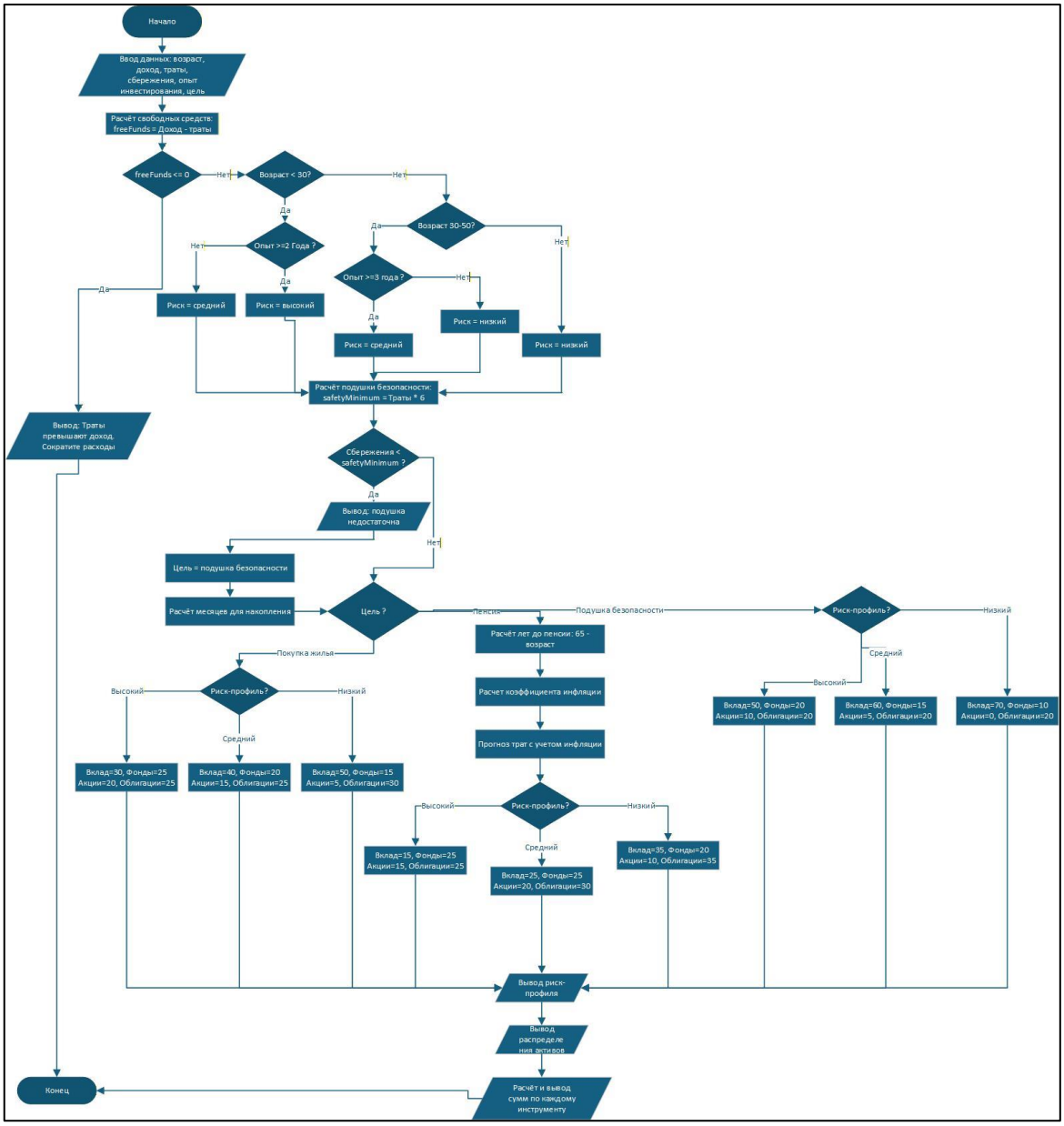
4) Проверить, достаточна ли "подушка безопасности", и если нет — перенаправить цель на ее формирование.

5) Учесть инфляцию в долгосрочных целях.

2. Контрольный пример.

Входные данные	**Значения**
age	Вводится пользователем
income	Вводится пользователем
expenses	Вводится пользователем
savings	Вводится пользователем
goal	1, 2 или 3
Выходные данные	**Значения**
free	Свободные средства (income - expenses)
risk	"высокий", "средний", "низкий"
Рекомендации	Распределение активов в %, срок формирования цели

3. Блок схема.


Начало
Ввод данных: возраст, доход, траты, сбережения, опыт инвестирования, цель
Расчёт свободных средств: freeFunds = Доход - траты
freeFunds <= 0
Нет
Да
Возраст < 30?
Нет
Да
Опыт >=2 Года ?
Нет
Да
Риск = средний
Риск = высокий
Возраст 30-50?
Да
Нет
Опыт >=3 года ?
Нет
Да
Риск = средний
Риск = низкий
Риск = низкий
Расчёт подушки безопасности: safetyMinimum = Траты * 6
Вывод: Траты превышают доход. Сократите расходы
Сбережения < safetyMinimum ?
Да
Вывод: подушка недостаточна
Нет
Цель = подушка безопасности
Расчёт месяцев для накопления
Цель ?
Пенсия
Подушка безопасности
Покупка жилья
Расчёт лет до пенсии: 65 - возраст
Расчет коэффициента инфляции
Прогноз трат с учетом инфляции
Риск-профиль?
Высокий
Средний
Низкий
Вклад=30, Фонды=25 Акции=20, Облигации=25
Вклад=40, Фонды=20 Акции=15, Облигации=25
Вклад=50, Фонды=15 Акции=5, Облигации=30
Риск-профиль?
Высокий
Средний
Низкий
Вклад=15, Фонды=25 Акции=15, Облигации=25
Вклад=25, Фонды=25 Акции=20, Облигации=30
Вклад=35, Фонды=20 Акции=10, Облигации=35
Риск-профиль?
Высокий
Средний
Низкий
Вклад=50, Фонды=20 Акции=10, Облигации=20
Вклад=60, Фонды=15 Акции=5, Облигации=20
Вклад=70, Фонды=10 Акции=0, Облигации=20
Вывод риск-профиля
Вывод распределения активов
Расчёт и вывод сумм по каждому инструменту
Конец

4. Текст программы.

```
int free = income - expenses;
Console.WriteLine($"\nСвободные средства: {free} руб/мес");

bool needPillow = (savings < expenses * 3);
if (needPillow)
{
    Console.WriteLine("\n⚠  Недостаточно подушки безопасности!");
    Console.WriteLine($"Рекомендуется: {expenses * 3} руб (3 месяца расходов)");
    Console.WriteLine("Сначала сформируйте подушку безопасности.");
    goal = 1;
}

string risk;
if (age < 30)
    risk = "высокий";
else if (age < 50)
    risk = "средний";
else
    risk = "низкий";

Console.WriteLine($"\nВаш риск-профиль: {risk}");

Console.WriteLine("\nРекомендуемое распределение активов:");

if (goal == 1)
{
    Console.WriteLine("Цель: Подушка безопасности");
    Console.WriteLine("Вклады: 100%");
    Console.WriteLine("Фонды: 0%");
    Console.WriteLine("Акции: 0%");
    Console.WriteLine("Облигации: 0%");
    Console.WriteLine("\nСрок формирования: " + (expenses * 3 - savings) / free + " месяцев");
}
else if (goal == 2)
{
    Console.WriteLine("Цель: Покупка жилья");
    if (risk == "высокий")
    {
        Console.WriteLine("Вклады: 40%");
        Console.WriteLine("Фонды: 30%");
        Console.WriteLine("Акции: 20%");
        Console.WriteLine("Облигации: 10%");
    }
    else if (risk == "средний")
    {
        Console.WriteLine("Вклады: 50%");
        Console.WriteLine("Фонды: 25%");
        Console.WriteLine("Акции: 10%");
        Console.WriteLine("Облигации: 15%");
    }
    else
    {
        Console.WriteLine("Вклады: 60%");
        Console.WriteLine("Фонды: 20%");
        Console.WriteLine("Акции: 5%");
        Console.WriteLine("Облигации: 15%");
    }
    Console.WriteLine("\n⚠Учтена инфляция ~5% годовых для сохранения покупательной способности.");
}
else
{
    Console.WriteLine("Цель: Пенсия");
    if (risk == "высокий")
    {
        Console.WriteLine("Вклады: 20%");
        Console.WriteLine("Фонды: 40%");
        Console.WriteLine("Акции: 30%");
        Console.WriteLine("Облигации: 10%");
    }
    else if (risk == "средний")
    {
        Console.WriteLine("Вклады: 30%");
        Console.WriteLine("Фонды: 35%");
        Console.WriteLine("Акции: 20%");
        Console.WriteLine("Облигации: 15%");
    }
    else
    {
        Console.WriteLine("Вклады: 40%");
        Console.WriteLine("Фонды: 30%");
        Console.WriteLine("Акции: 10%");
        Console.WriteLine("Облигации: 20%");
    }
    Console.WriteLine("\n⚠Учтена инфляция ~5% годовых для долгосрочного роста.");
}

Console.WriteLine("\nРекомендация: откладывайте " + free + " руб ежемесячно.");
```

1. Постановка задачи.

Спроектируйте блок-схему ядра программы, которая обрабатывает команды для управления умным домом из единой очереди сообщений.

Команды: device.action(param), например, light.kitchen.on, thermostat.set(22), blinds.open(50%).

Алгоритм должен:

Парсить строку команды.

Проверять существование устройства и допустимость действия.

Проверять безопасность (не включить духовку на ночь).

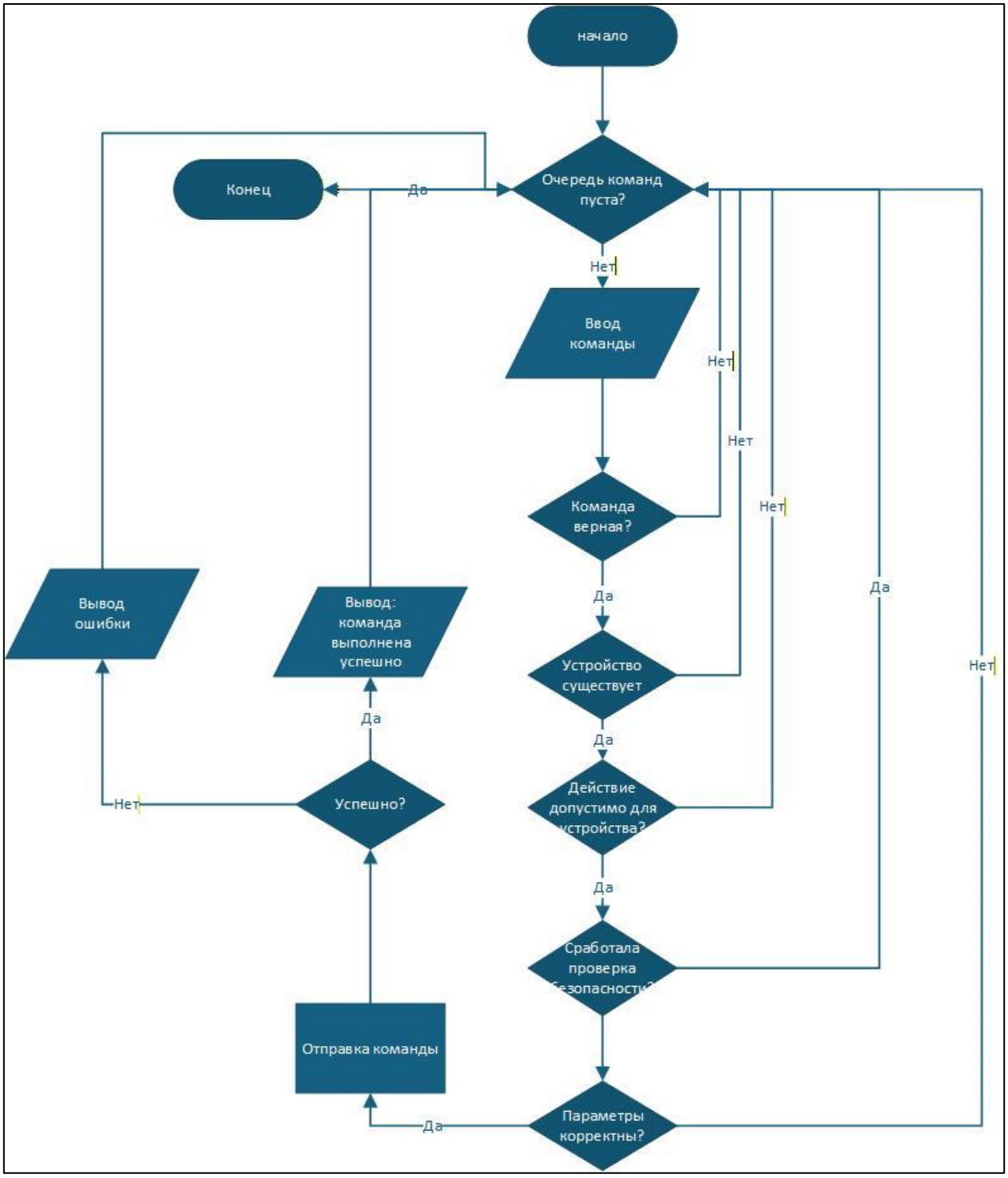
Отправлять команду устройству и обрабатывать статус ответа (успех/ошибка).

Логировать все действия.

2. Контрольный пример.

Входные данные	**Значения**
cmd	"light.kitchen.on", "oven.on", "thermostat.set(22)", "exit"
validDevices	"light", "thermostat", "blinds", "oven"
validActions	"on", "off", "set", "open", "close"
Выходные данные	**Значения**
Log	"[LOG] Получена команда...", "[LOG] Ошибка...", "[LOG] Выполнение...", "[LOG] Статус: УСПЕХ"

3. Блок схема.


начало
Очередь команд пуста?
Конец
Да
Нет
Ввод команды
Нет
Нет
Нет
Команда верная?
Да
Да
Устройство существует
Нет
Да
Действие допустимо для устройства?
Да
Сработала проверка безопасности?
Параметры корректны?
Да
Нет
Отправка команды
Успешно?
Да
Нет
Вывод: команда выполнена успешно
Вывод ошибки

4. Текст программы.

```
while (true)
{
    Console.Write("\nВведите команду (exit для выхода): ");
    string cmd = Console.ReadLine().Trim();

    if (cmd.ToLower() == "exit")
        break;

    Console.WriteLine($"[LOG] Получена команда: {cmd}");

    if (string.IsNullOrEmpty(cmd))
    {
        Console.WriteLine("[LOG] Ошибка: пустая команда");
        continue;
    }

    string device = "", action = "", param = "";
    bool parseOk = false;

    if (cmd.Contains("."))
    {
        string[] parts = cmd.Split('.');
        if (parts.Length >= 2)
        {
            device = parts[0].ToLower();
            string actPart = parts[1];

            if (actPart.Contains("(") && actPart.Contains(")"))
            {
                int open = actPart.IndexOf('(');
                int close = actPart.IndexOf(')');
                action = actPart.Substring(0, open).ToLower();
                param = actPart.Substring(open + 1, close - open - 1);
            }
            else
            {
                action = actPart.ToLower();
            }
            parseOk = true;
        }
    }

    if (!parseOk)
    {
        Console.WriteLine("[LOG] Ошибка парсинга: неверный формат команды");
        continue;
    }

    bool devOk = false, actOk = false;
    foreach (string d in validDevices) if (d == device) devOk = true;
    foreach (string a in validActions) if (a == action) actOk = true;

    if (!devOk)
    {
        Console.WriteLine($"[LOG] Ошибка: неизвестное устройство '{device}'");
        continue;
    }
    if (!actOk)
    {
        Console.WriteLine($"[LOG] Ошибка: недопустимое действие '{action}'");
        continue;
    }

    DateTime now = DateTime.Now;
    bool isNight = (now.Hour >= 23 || now.Hour < 6);

    if (device == "oven" && action == "on" && isNight)
    {
        Console.WriteLine("[LOG] БЕЗОПАСНОСТЬ: Запрет включения духовки ночью!");
        continue;
    }

    Console.WriteLine($"[LOG] Выполнение: {device}.{action}({param})");
    Console.WriteLine($"[LOG] Статус: УСПЕХ - команда применена");
    Console.WriteLine($"[LOG] Время: {now:HH:mm:ss}");
}

Console.WriteLine("\nРабота интерпретатора завершена.");
```

1. Постановка задачи.

Разработайте ИИ для компьютерного противника в пошаговой стратегии.

На каждом ходе алгоритм должен:

1) Оценить текущую карту (ресурсы, свои и вражеские юниты, укрепления).

2) Выбрать глобальную стратегию ("атака", "оборона", "развитие экономики") на основе сравнения сил.

3) В режиме "атака": найти самый слабый/ценный вражеский объект в радиусе досягаемости, сформировать ударную группу, отдать приказ.

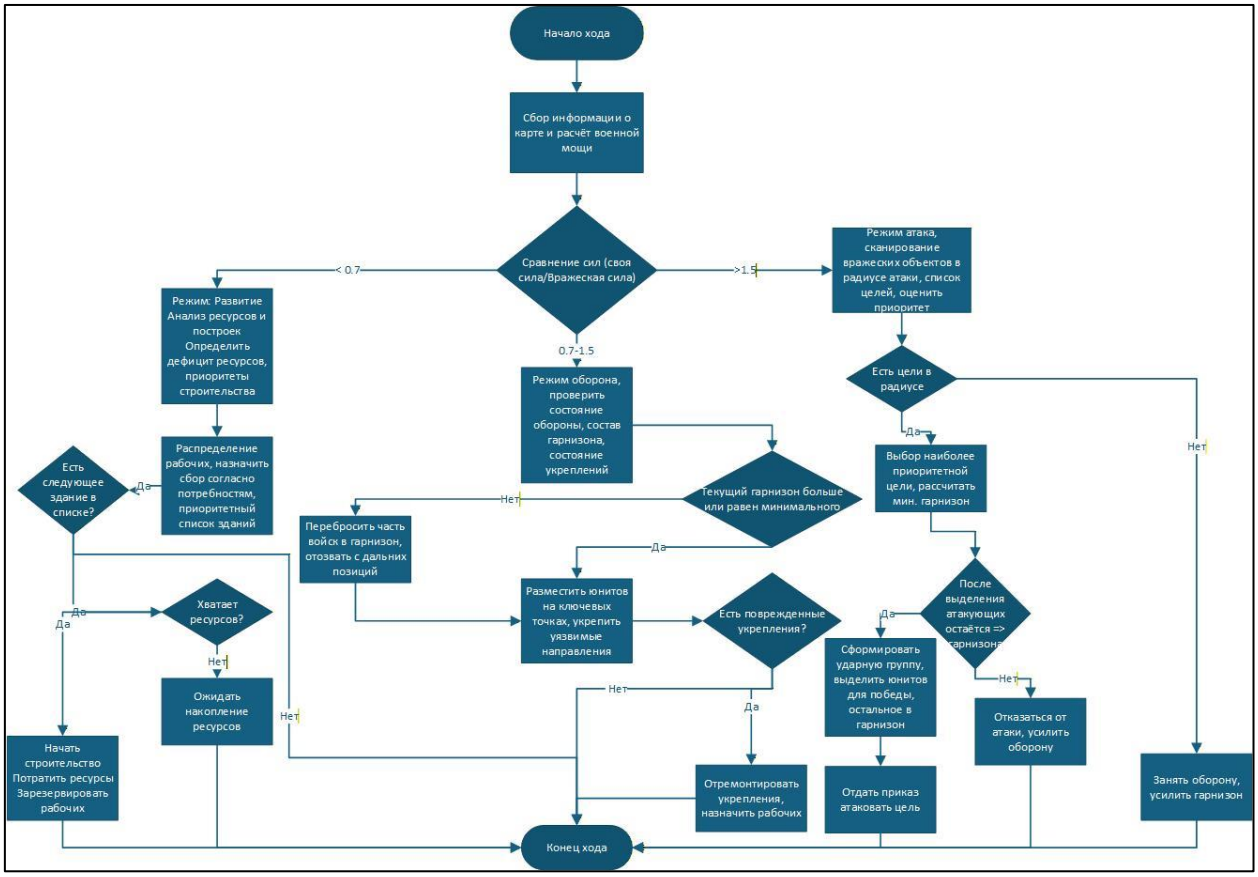
4) В режиме "развитие": распределить рабочих между ресурсами, строить новые здания по приоритетному списку.

5) Всегда оставлять минимальный гарнизон для обороны базы.

2. Контрольный пример.

Входные данные	**Значения**
myUnits, enemyUnits	12, 15
myWorkers, resources	6, 400
baseDefenseMin	4
Выходные данные	**Значения**
strategy	"атака", "оборона", "развитие"
Состояние	Обновленные значения юнитов, ресурсов, рабочих

3. Блок схема.


Начало хода
Сбор информации о карте и расчёт военной мощи
Сравнение сил (своя сила/Вражеская сила)
< 0.7
0.7-1.5
>1.5
Режим: Развитие Анализ ресурсов и построек Определить дефицит ресурсов, приоритеты строительства
Распределение рабочих, назначить сбор согласно потребностям, приоритетный список зданий
Есть следующее здание в списке?
Да
Хватает ресурсов?
Нет
Ожидать накопление ресурсов
Начать строительство Потратить ресурсы Зарезервировать рабочих
Режим оборона, проверить состояние обороны, состав гарнизона, состояние укреплений
Текущий гарнизон больше или равен минимального
Перебросить часть войск в гарнизон, отозвать с дальних позиций
Разместить юнитов на ключевых точках, укрепить уязвимые направления
Есть поврежденные укрепления?
Отремонтировать укрепления, назначить рабочих
Режим атака, сканирование вражеских объектов в радиусе атаки, список целей, оценить приоритет
Есть цели в радиусе
Выбор наиболее приоритетной цели, рассчитать мин. гарнизон
После выделения атакующих остаётся => гарнизона
Сформировать ударную группу, выделить юнитов для победы, остальное в гарнизон
Отдать приказ атаковать цель
Отказаться от атаки, усилить оборону
Занять оборону, усилить гарнизон
Конец хода

4. Текст программы.

```
while(enemyUnits >= 0)
{
    Console.WriteLine($"=== ХОД {turn} ===");
    Thread.Sleep(1000);
    Console.WriteLine($"Состояние: юниты={myUnits}, рабочие={myWorkers}, ресурсы={resources}\n");
    Thread.Sleep(1000);

    string strategy;
    if (myUnits >= enemyUnits + 5)
        strategy = "атака";
    else if (myUnits <= enemyUnits - 5 || myUnits < baseDefenseMin)
        strategy = "оборона";
    else
        strategy = "развитие";

    Console.WriteLine($"Стратегия: {strategy}");
    Thread.Sleep(1000);

    if (strategy == "атака")
    {
        int attackForce = myUnits - baseDefenseMin;
        if (attackForce < 1) attackForce = 1;
        int targetValue = 0;
        Console.WriteLine($"Атака: сформирован отряд из {attackForce} юнитов");
        Thread.Sleep(1000);
        Console.WriteLine($"Цель: вражеский склад (ценность={targetValue}, слабая охрана)");
        Thread.Sleep(1000);
        Console.WriteLine($"Гарнизон базы: {baseDefenseMin} юнитов");
        myUnits -= attackForce / 3;
        enemyUnits -= attackForce / 2;
        resources += 100;
    }
    else if (strategy == "оборона")
    {
        int defenseNow = myUnits;
        if (defenseNow < baseDefenseMin)
        {
            Console.WriteLine($"Недостаток обороны! Требуется минимум {baseDefenseMin} юнитов.");
            Thread.Sleep(1000);
            Console.WriteLine("Приоритет: удержание базы, сбор ресурсов приостановлен.");
        }
        else
        {
            Console.WriteLine($"Оборона: {defenseNow} юнитов на базе");
            Thread.Sleep(1000);
            Console.WriteLine("Рабочие: 50% сбор ресурсов, 50% ремонт укреплений");
            resources += 30;
        }
    }
    else
    {
        Console.WriteLine("Развитие экономики:");
        Thread.Sleep(1000);
        Console.WriteLine($"Рабочие: {myWorkers / 2} → дерево, {myWorkers / 2} → руда");
        Thread.Sleep(1000);
        resources += myWorkers * 15;

        if (resources >= 250)
        {
            Console.WriteLine("Строительство: казарма (250 ресурсов)");
            Thread.Sleep(1000);
            resources -= 250;
            myUnits += 5;
        }
        else if (resources >= 150)
        {
            Console.WriteLine("Строительство: ферма (150 ресурсов)");
            Thread.Sleep(1000);
            resources -= 150;
            myWorkers += 2;
        }

        Console.WriteLine($"Гарнизон базы: {baseDefenseMin} юнитов (резерв)");
    }

    Console.WriteLine($"\nИтог хода {turn}:");
    Console.WriteLine($"Юниты: {myUnits}, Рабочие: {myWorkers}, Ресурсы: {resources}");
    Thread.Sleep(1000);
    Console.WriteLine($"Враг: {enemyUnits} юнитов\n");
    Thread.Sleep(3000);
    turn++;
}

Console.WriteLine("Игра завершена.");
```